

A Comprehensive Execution Protocol for a Comparative Study of MIL Architectures in Computational Pathology

Part I: Strategic & Environmental Foundations

1.1 The Feasibility Mandate: Validating the Research Pivot

The foundational premise of this research project is a strategic pivot away from a computationally infeasible end-to-end pipeline toward a focused, rigorous, and achievable modeling problem. Modern computational pathology workflows for analyzing gigapixel-scale Whole Slide Images (WSIs) are typically divided into two stages: feature extraction and prognostic modeling. The first stage, feature extraction, is the primary computational bottleneck, demanding significant GPU VRAM (often 8 GB or more) and hundreds of processing hours to convert raw image data into numerical embeddings. These requirements far exceed the capabilities of the specified hardware, an NVIDIA 1650 with 4 GB of VRAM, rendering any attempt to train or even run state-of-the-art feature extractors like Vision Transformers (ViTs) or foundation models (e.g., UNI, CONCH) impossible.

This technical constraint, however, enables a more methodologically robust experimental design. By leveraging publicly available, pre-computed feature sets generated by well-resourced research groups, the project bypasses the entire resource-intensive feature extraction stage. This approach aligns with an emerging and productive paradigm in the field, where institutions like the Mahmood Lab at Harvard act as "feature producers," democratizing access to high-quality data representations. This pivot reframes the research from a broad engineering challenge into a focused scientific inquiry. By using a single, fixed, state-of-the-art feature set (UNI2-h), the study eliminates the feature extractor as a confounding variable. This creates a controlled environment where any observed performance differences between the Multiple Instance Learning (MIL) models can be attributed solely to the architectural differences in their aggregation mechanisms. This constraint thus transforms a potential weakness into a key methodological strength, enabling a scientifically pure "apples-to-apples" comparison that is often difficult to achieve in end-to-end studies.

1.2 Curating the Digital Laboratory: Environment Setup in VS Code

A reproducible computational environment is the bedrock of credible scientific research. This section provides a granular, command-by-command protocol for establishing this environment on the target machine using Visual Studio Code (VS Code).

Anaconda and Conda Environment The use of a dedicated virtual environment is mandatory to isolate project dependencies and ensure that the research can be perfectly replicated in the future. Anaconda is the recommended tool for this purpose.

1. **Create the Conda Environment:** Open an integrated terminal in VS Code (or any

system terminal) and execute the following command. This creates a new environment named `mil_thesis` with Python version 3.8, which is compatible with the required libraries.

```
conda create -n mil_thesis python=3.8 -y
```

2. **Activate the Environment:** Before installing any packages, activate the newly created environment. This command must be run in every new terminal session dedicated to this project.

```
conda activate mil_thesis
```

NVIDIA Driver and PyTorch Installation Proper installation of PyTorch with CUDA support is critical for leveraging the NVIDIA 1650 GPU.

1. **Verify NVIDIA Driver:** Ensure that the system's NVIDIA drivers are up-to-date. This can be checked via the NVIDIA Control Panel under "System Information". Modern drivers are required for compatibility with the CUDA runtime packaged with PyTorch.
2. **Install PyTorch with CUDA:** The PyTorch binaries ship with their own self-contained CUDA runtime, cuDNN, and other necessary libraries. This means it is **not** necessary to perform a system-wide installation of the NVIDIA CUDA Toolkit. The installation command should be obtained directly from the official PyTorch website to ensure compatibility. For the hardware in question, a version built with CUDA 11.8 is appropriate. Execute the following command within the activated `mil_thesis` environment:

```
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu118
```

3. **Verify Installation:** After installation, confirm that PyTorch can detect the GPU. Run the following command in a Python interpreter within the environment:

```
import torch
print(f"CUDA Available: {torch.cuda.is_available()}")
print(f"Device Name: {torch.cuda.get_device_name(0) if
torch.cuda.is_available() else 'N/A'}")
```

The output should confirm that CUDA is available and list the GeForce GTX 1650.

Core Library Installation Install the remaining core dependencies for data manipulation, machine learning, and visualization using a single pip command.

```
pip install pandas scikit-learn h5py opencv-python-headless matplotlib
seaborn pytorch-lightning lifelines scikit-survival MLstatkit
```

Table 1: Environment Dependency Manifest For absolute reproducibility, the following table lists the key packages and their recommended versions. These can be installed explicitly (e.g., `pip install pandas==2.0.3`) or captured after installation by running `pip freeze > requirements.txt`.

Package	Version	Purpose
python	3.8.x	Core programming language
torch	2.1.0+cu118	Deep learning framework
torchvision	0.16.0+cu118	PyTorch computer vision utilities
pandas	2.0.3	Data manipulation and analysis
scikit-learn	1.3.2	Core machine learning utilities
h5py	3.10.0	Reading/writing HDF5 files
pytorch-lightning	2.1.0	High-level PyTorch wrapper

Package	Version	Purpose
lifelines	0.27.8	Survival analysis (Log-rank test)
scikit-survival	0.22.1	Survival analysis models and metrics
MLstatkit	1.0.5	Statistical tests (DeLong's test)
matplotlib	3.7.3	Plotting and visualization
seaborn	0.13.0	Statistical data visualization

1.3 The AI Research Assistant: Integrating Gemini in VS Code

Integrating a Large Language Model (LLM) as a research assistant can significantly accelerate development, debugging, and documentation. This project will leverage Gemini and its command-line interface (CLI) directly within the VS Code terminal.

Installation and Setup

1. **Install the Extension:** Open the Extensions view in VS Code (Ctrl+Shift+X), search for "Gemini Code Assist," and install it. Follow the prompts to sign in with a Google Account.
2. **Integrate Gemini CLI:** The Gemini CLI can be connected to the VS Code IDE to gain awareness of the workspace, open files, and selected text. This enables highly contextual and powerful interactions. From the integrated terminal in VS Code, run the following one-time setup commands:

```
# Install the necessary companion extension from the CLI
/ide install
```

```
# Enable the integration for the current session
/ide enable
```

Defining the Scope of LLM Assistance The effectiveness of an LLM assistant hinges on assigning it discrete, well-defined tasks. Rather than asking broad questions like "write the code for my thesis," the LLM should be used as a targeted tool for specific sub-problems. For this project, Gemini's role will be focused on:

- **Code Generation:** Creating boilerplate code for specific, well-understood tasks, such as the HDF5 data loading snippet.
- **Code Explanation:** Providing clear, natural-language explanations of complex functions or classes within the existing CLAM and TransMIL codebases.
- **Debugging:** Assisting in diagnosing and resolving errors by providing the error message and the surrounding code context.
- **Documentation:** Generating professional docstrings for functions and clear Markdown explanations for reporting purposes.

This approach treats the LLM as a powerful pair-programmer and research aide, enhancing productivity without supplanting the core intellectual work of the researcher.

Part II: The Data Acquisition and Curation Pipeline

2.1 Gaining Access to State-of-the-Art Features: The Hugging Face Protocol

The project's success depends on obtaining the pre-computed WSI features from the Mahmood Lab. This dataset, MahmoodLab/UNI2-h-features, is hosted on the Hugging Face platform and is subject to gated access to ensure responsible use.

The access control is not merely an administrative hurdle; it is a critical component of the data governance framework for sensitive medical research data. The features are derived from patient tissue samples, and the gate serves to enforce the terms of the CC-BY-NC-ND 4.0 license, which restricts use to non-commercial, academic research and helps prevent the re-identification of patient data. Understanding this context is crucial for any researcher working with such datasets.

To request access:

1. Navigate to the dataset page:
<https://huggingface.co/datasets/MahmoodLab/UNI2-h-features>.
2. Create a Hugging Face account or log in.
3. **Crucially, ensure the primary email address associated with the Hugging Face account is a verified institutional email (e.g., @university.edu).** Requests from personal email accounts (@gmail.com, etc.) will be automatically denied.
4. Click the button to request access and agree to the terms of use, attesting that the data will be used for non-commercial academic research.
5. Once access is granted, download the compressed archive for the TCGA-BRCA cohort. The data is distributed as .h5 files, each containing feature embeddings and their spatial coordinates.

2.2 Sourcing Clinical Ground Truth: Navigating the GDC PanCanAtlas

The second essential data component is the clinical outcome data. To ensure the highest quality and consistency for survival analysis, the officially curated TCGA Pan-Cancer Clinical Data Resource (TCGA-CDR) is the mandated source. This resource was specifically created to standardize heterogeneous clinical data from across the TCGA project, making it superior to raw clinical downloads for robust outcome analytics.

The specific file required is TCGA-CDR-SupplementalTableS1.xlsx. It can be located and downloaded from the Genomic Data Commons (GDC) website:

1. Navigate to the GDC's main page for TCGA PanCanAtlas Publications.
2. Locate the section listing data files.
3. Find the entry for "TCGA-Clinical Data Resource (CDR) Outcome" and download the corresponding .xlsx file. This file is open access and does not require a special token.

2.3 Forging the Master Dataset: The master.csv

The final preparatory step is to merge the feature file information and the clinical data into a single, analysis-ready master.csv file. This file will serve as the sole manifest for all subsequent training and evaluation scripts. The process involves using the pandas library in Python.

1. **Load Clinical Data:** Load the downloaded TCGA-CDR-SupplementalTableS1.xlsx into a pandas DataFrame.
2. **Process Outcome Variables:** Identify the columns for Overall Survival (OS). The key columns are OS.time (survival time in days) and OS (censoring status, where 1 typically indicates a deceased patient and 0 indicates a living/censored patient).
3. **Generate Feature File Manifest:** Write a script to iterate through the directory containing the downloaded .h5 feature files. For each file, extract the slide_id, which is the filename

without the .h5 extension (e.g., TCGA-A1-AOSD-01A-01-TS1.h5 becomes TCGA-A1-AOSD-01A-01-TS1).

4. **Merge DataFrames:** The patient identifier in the TCGA-CDR file (often bcr_patient_barcode) corresponds to the first 12 characters of the slide_id. Create a new column in both the clinical DataFrame and the slide list DataFrame containing this 12-character patient ID. Perform a merge operation on this common key to link each slide to its corresponding patient's clinical outcome data.
5. **Finalize Schema:** Ensure the final master.csv file has the required columns as specified in the thesis plan: case_id, slide_id, OS_time, and OS_status. It is also recommended to create a binarized label for 5-year survival (survival_label_5yr) for classification tasks, where the value is 1 if OS_status is 1 and OS.time is less than 1825 days, and 0 otherwise.

Part III: Project Architecture & Workflow Management

3.1 Structuring Your Research Workspace: A Modular, Two-Repository Approach

A common question for projects involving multiple existing codebases is how to structure the workspace. Merging the CLAM and TransMIL repositories into a single project folder is strongly discouraged. A far more robust and professional approach is to adopt a modular, multi-repository structure that enforces a clean separation of concerns.

This separation is not just about file organization; it is a core principle of reproducible research. CLAM and TransMIL represent two distinct, independent scientific implementations. Mixing their codebases creates unnecessary dependencies, introduces the risk of breaking the original, validated code, and makes it difficult to update one framework without impacting the other. The recommended structure treats each model as a distinct "experiment" that consumes standardized "reagents" (the data and split files).

The recommended directory structure is as follows:

```
MIL_Thesis_Project/
├── data/
│   ├── TCGA-BRCA-features/  # All.h5 files are placed here
│   └── master.csv           # The master data manifest
├── splits/
│   └── 5-fold-splits/       # Generated split files are stored here
├── CLAM/                   # Result of 'git clone
https://github.com/mahmoodlab/CLAM.git'
└── TransMIL/               # Result of 'git clone
https://github.com/szc19990412/TransMIL.git'
```

This architecture ensures that the data and the models are decoupled. The CLAM and TransMIL projects remain pristine clones of their official repositories, with only minimal, targeted changes made (e.g., to the data loader). All data and experimental artifacts (like cross-validation splits) are stored externally, accessible to both model frameworks.

3.2 Managing Data and Splits: The Single Source of Truth

For a comparative study to be valid, both models must be trained and evaluated on the exact same data partitions. Therefore, a single, canonical set of cross-validation splits must be generated and used for all experiments.

The CLAM repository provides a convenient utility script, `create_splits_seq.py`, for this purpose. This script should be run once, pointing to the `master.csv` file. It will generate `.csv` files for each of the k-folds (e.g., 5-fold cross-validation), ensuring that slides from the same patient (`case_id`) are kept together within a single fold and do not leak between training and testing sets.

The output of this script should be saved to the central `splits/5-fold-splits/` directory. Both the CLAM and TransMIL training pipelines will be configured to read their fold definitions from this single source of truth. This is possible because the TransMIL framework explicitly follows the data processing conventions established by CLAM, making the splits directly compatible.

Part IV: Implementing the Baseline: A Deep Dive into CLAM

4.1 Codebase Preparation and Split Generation

1. **Clone the Repository:** Navigate to the main `MIL_Thesis_Project/` directory in the terminal and clone the official CLAM repository.

```
git clone https://github.com/mahmoodlab/CLAM.git
```
2. **Generate Splits:** As described in section 3.2, use the provided script to generate the 5-fold cross-validation files. The command will look similar to this, executed from within the `CLAM/` directory:

```
python create_splits_seq.py --task survival --seed 1 --label_col OS_status --k 5 --data_dir ../data/master.csv
```

The output should be directed to the central `MIL_Thesis_Project/splits/` folder.
3. **Note on Workflow:** It is critical to remember that the project workflow begins directly at the model training stage (`main.py`). The data preprocessing and feature extraction scripts within the CLAM repository (`create_patches_fp.py`, `extract_features_fp.py`) must be ignored, as this stage has been strategically bypassed.

4.2 The Critical Adaptation: Modifying the Data Loader for HDF5

This is the most significant code modification required. The default CLAM data loader is designed to read feature files in PyTorch's `.pt` format. It must be adapted to handle the `.h5` files from the UNI2-h dataset.

1. **Locate the Target File:** Open the file `CLAM/datasets/dataset_generic.py` in VS Code.
2. **Identify the Method:** Find the `__getitem__` method within the `Generic_MIL_Dataset` class.
3. **Modify the Code:** The original code contains a line similar to `features = torch.load(full_path)`. This line must be replaced with a code block that uses the `h5py` library.

The replacement code is as follows:

```
# Add this import at the top of the file
import h5py
```

```
#... inside the __getitem__ method of Generic_MIL_Dataset...
```

```
# Replace the original torch.load line with this block
with h5py.File(full_path, 'r') as hf:
    features = torch.from_numpy(hf['features'][:])
```

This is an ideal task for the integrated Gemini assistant. One can highlight the original torch.load line and use a prompt in the terminal like: "In my VS Code terminal, using Gemini CLI, generate a Python code snippet to replace the active selection. The new code should use the h5py library to open the file at path 'full_path', read the dataset named 'features', and return it as a PyTorch tensor."

4.3 Executing the Training Protocol

With the data loader adapted, the training pipeline can be executed. A single command launches the process for one fold of the cross-validation. The command should be run from the CLAM/ directory.

```
CUDA_VISIBLE_DEVICES=0 python main.py
--data_root_dir../data/TCGA-BRCA-features/
--csv_path../data/master.csv --task brca_survival --exp_code
brca_survival_clam_fold_0 --model_type clam_sb --bag_loss nll_surv
--splits_dir../splits/5-fold-splits/ --lr 2e-4 --k 5
```

To automate running all 5 folds, a simple bash loop can be used:

```
for i in {0..4}
do
    echo "Running Fold $i"
    CUDA_VISIBLE_DEVICES=0 python main.py
--data_root_dir../data/TCGA-BRCA-features/
--csv_path../data/master.csv --task brca_survival --exp_code
brca_survival_clam_fold_$i --model_type clam_sb --bag_loss nll_surv
--splits_dir../splits/5-fold-splits/ --lr 2e-4 --k 5 --split $i
done
```

Table 2: CLAM Training Command Reference The following table deconstructs the key arguments in the training command for clarity and future modification.

Argument	Example Value	Description
--data_root_dir	../data/TCGA-BRCA-features/	Path to the directory containing the .h5 feature files.
--csv_path	../data/master.csv	Path to the master CSV file linking slides to labels.
--splits_dir	../splits/5-fold-splits/	Path to the directory containing the generated split files.
--task	brca_survival	A custom task name defined to load the correct dataset configuration.

Argument	Example Value	Description
--exp_code	brca_survival_clam_fold_0	A unique name for this experiment. Results will be saved in results/.
--model_type	clam_sb	Specifies the CLAM model variant (single-branch). Use clam_mb for multi-branch.
--bag_loss	nll_surv	Specifies the loss function for survival analysis (Negative Log Likelihood).
--lr	\$2e-4\$	Learning rate for the optimizer.
--k	5	The total number of folds for cross-validation.
--split	0	The specific fold to run for this execution (used in the loop).

Part V: Implementing the Challenger: Adapting the TransMIL Framework

5.1 Leveraging Prior Work

The implementation of TransMIL is significantly accelerated because the most labor-intensive steps—data acquisition, curation, and split generation—are already complete. The master.csv and the split files generated in the previous steps are directly reusable for the TransMIL experiments, as its data handling is designed to be compatible with the CLAM pipeline.

5.2 Configuration Over Code: Crafting the YAML File

TransMIL uses YAML files for experiment configuration, which is a best practice for separating hyperparameters from the core model code. A new configuration file, TransMIL_survival.yaml, must be created inside a new directory, for instance, TransMIL/TCGA-BRCA/.

This file defines all necessary paths and hyperparameters. It is critical to keep parameters like the learning rate consistent with the CLAM experiments to ensure a fair and direct comparison.

Table 3: TransMIL YAML Configuration Reference The following template provides the necessary structure and values for the TransMIL_survival.yaml file.

Key Path	Example Value	Description
data.root	../data/TCGA-BRCA-features/	Relative path from TransMIL/ to the feature files directory.
data.csv_path	../data/master.csv	Relative path to the master CSV file.
data.splits_path	../splits/5-fold-splits/	Relative path to the directory containing the split files.
model.n_classes	2	Number of output classes (for binary 5-year survival). For direct risk prediction, this might be set to 1.

Key Path	Example Value	Description
model.loss_fn	nll_surv	Specifies the survival loss function, consistent with CLAM.
optimizer.lr	\$2e-4\$	Learning rate, kept consistent with CLAM for a fair comparison.
training.max_epochs	200	Maximum number of training epochs.
training.gpus	``	Specifies which GPU device(s) to use.

5.3 The HDF5 Data Loader Adaptation (TransMIL)

Similar to CLAM, the default TransMIL data loader must be modified to read .h5 files instead of .pt files. The process is identical:

1. **Clone the Repository:**

```
git clone https://github.com/szc19990412/TransMIL.git
```

2. **Locate the Data Loader:** Inspect the TransMIL codebase to find the relevant data loading script (e.g., a file named datasets.py or similar).

3. **Identify the Loading Function:** Find the method responsible for loading a single slide's features (likely within a `__getitem__` method).

4. **Apply the h5py Snippet:** Replace the existing file loading line with the same h5py code block used for CLAM:

```
import h5py
import torch
#...
with h5py.File(full_path, 'r') as hf:
    features = torch.from_numpy(hf['features'][:])
```

5.4 Launching the TransMIL Experiments

With the configuration file and data loader in place, training is initiated with a simpler command, as most parameters are now defined in the YAML file.

```
python train.py --stage='train'
--config='TCGA-BRCA/TransMIL_survival.yaml' --gpus=0 --fold=0
```

An analogous bash script loop should be used to systematically run training for all 5 folds:

```
for i in {0..4}
do
    echo "Running Fold $i"
    python train.py --stage='train'
--config='TCGA-BRCA/TransMIL_survival.yaml' --gpus=0 --fold=$i
done
```

Due to its more complex self-attention mechanism, TransMIL's training may take slightly longer

per epoch than CLAM, but it should remain well within the capabilities of the NVIDIA 1650 GPU.

Part VI: Rigorous Evaluation & Scientific Inquiry

6.1 Quantitative Synthesis: Collating Results

After all training folds for both models are complete, the first step in analysis is to collate the quantitative performance metrics from the generated logs or result files. For each of the 5 test folds, extract the primary evaluation metrics:

- **Concordance Index (C-Index):** The primary metric for survival analysis, measuring the model's ability to correctly rank patients by predicted risk. A C-Index of 1.0 is perfect, while 0.5 is equivalent to random chance.
- **Area Under the ROC Curve (AUC):** The primary metric for the binary 5-year survival classification task, measuring the model's ability to discriminate between the two classes.

Calculate the mean and standard deviation for each metric across the 5 folds. This provides a robust estimate of each model's performance and its stability.

Table 4: Final Comparative Performance Matrix (Template) The final results should be presented in a clear, professional table, ready for inclusion in the thesis document.

Model	Metric	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean \pm Std. Dev.
CLAM	C-Index						
	AUC (5-yr)						
TransMIL	C-Index						
	AUC (5-yr)						

6.2 The Litmus Test of Significance: Statistical Comparison

Observing a difference in the mean performance metrics is not sufficient for a rigorous scientific claim. It must be demonstrated that the observed difference is statistically significant and not merely due to random chance from a particular data split. Two key statistical tests are required for this thesis.

Comparing Survival Curves (Log-rank Test) This test determines if a model can successfully stratify patients into groups with statistically different survival outcomes. The null hypothesis is that the survival curves of the groups are identical.

1. For a given model and test fold, take the predicted risk scores for all patients.
2. Divide the patients into a "high-risk" group and a "low-risk" group based on the median predicted risk score.
3. Using a library like lifelines or scikit-survival, perform a log-rank test comparing the survival data (OS_time, OS_status) of the high-risk group versus the low-risk group.
4. A resulting p-value of less than 0.05 ($p < 0.05$) indicates that the model has learned a meaningful prognostic signal and can successfully separate patients into groups with significantly different survival outcomes.

Comparing AUCs (DeLong's Test) This test is the standard method for comparing the AUCs of two models evaluated on the same set of test data. It correctly accounts for the correlated nature of the predictions. The null hypothesis is that the true AUCs of the two models are identical.

1. For a given test fold, gather the true binary labels (e.g., survival_label_5yr) and the

predicted probabilities from both CLAM and TransMIL.

2. Use a well-vetted implementation of DeLong's test, such as the one available in the MLstatkit library.
3. A resulting p-value of less than 0.05 ($p < 0.05$) indicates that the observed difference in AUC between CLAM and TransMIL is statistically significant.

Table 5: Statistical Test Selection Guide

Research Question	Statistical Test	Null Hypothesis (H_0)	Interpretation of $p < 0.05$	Python Library
Does my model effectively separate patients into high- and low-risk survival groups?	Log-rank Test	The survival distributions of the stratified groups are identical.	The model's risk stratification is statistically significant.	lifelines, scikit-survival
Is the classification performance (AUC) of Model A significantly different from Model B?	DeLong's Test	The AUC of Model A is equal to the AUC of Model B.	The difference in AUC between the two models is statistically significant.	MLstatkit

6.3 Qualitative Analysis and Interpretability: Generating Heatmaps

Quantitative results demonstrate *what* a model can do; qualitative analysis provides insight into *how* and *why*. Both CLAM and TransMIL are attention-based models, meaning they produce attention scores that indicate which image patches (instances) were most influential in making a slide-level prediction.

These attention scores can be visualized as heatmaps overlaid on a thumbnail of the WSI. The visualization scripts provided in the CLAM repository can be used as a starting point. These scripts will need to be adapted to read the attention scores saved during TransMIL's evaluation phase to enable a direct, side-by-side comparison.

To generate a compelling narrative for the thesis, a structured selection of cases for visualization is recommended :

- A case where both models correctly predicted a high-risk outcome.
- A case where both models failed to predict a poor outcome.
- A discordant case, where one model succeeded and the other failed.

By juxtaposing the heatmaps for these cases, the analysis can generate hypotheses about the models' decision-making processes. For instance, one might observe that TransMIL focuses more on the tumor-stroma interface while CLAM attends to regions of high cellular density. This links the computational mechanisms back to underlying histopathology, providing a much richer interpretation of the results.

Part VII: The AI-Assisted Researcher: A Workflow-Integrated Guide to Gemini

7.1 A Library of High-Impact Prompts

The following is a curated list of prompts for the Gemini CLI, tailored to the specific workflow of this project. These can be used directly in the VS Code terminal.

- **Code Generation & Refactoring:**
 - gemini write a python function using pandas to load a csv file at a given path, extract the 'slide_id' and 'OS_time' columns, and return them as a dictionary.
 - gemini refactor this for loop to use a more efficient pandas vectorized operation and add comments explaining the change.
- **Debugging:**
 - gemini explain this PyTorch error: "CUDA out of memory". My GPU has 4GB VRAM. Suggest three specific ways to reduce VRAM usage in my training script without changing the model architecture.
 - gemini I'm getting a "FileNotFoundError" when my script tries to load data. The path seems correct. Here is my code: [paste code]. What are common reasons for this error in this context?
- **Documentation & Reporting:**
 - gemini write a professional docstring for this python function. Explain its parameters, what it returns, and any exceptions it might raise.
 - gemini explain the concept of a Concordance Index in survival analysis in simple but precise terms for the introduction of a master's thesis.
 - gemini summarize the key architectural difference between an attention mechanism like in CLAM and a self-attention mechanism in a Transformer like TransMIL.

7.2 A Day-by-Day Integration Plan

This plan maps the prompts onto the 7-day prototyping sprint outlined in the thesis plan , demonstrating when and how to leverage the AI assistant effectively.

- **Day 1-2 (Environment & Data Curation):** Use documentation prompts to understand the structure of the TCGA-CDR data. Use code generation prompts for parts of the master.csv creation script.
- **Day 3 (CLAM Adaptation):** Use the specific code generation prompt for the HDF5 data loader adaptation (as described in section 4.2).
- **Day 4 (CLAM Training):** Use debugging prompts to troubleshoot any CUDA out of memory or data loading errors that may arise during the initial training runs.
- **Day 5-6 (TransMIL Implementation):** Use code explanation prompts to quickly understand the TransMIL data loading pipeline before adapting it.
- **Day 7 (Synthesis & Analysis):** Use reporting prompts to help draft clear explanations of the C-Index, AUC, and the log-rank test for the final analysis report. Use code generation prompts to script the collation of results from log files.

Part VIII: From Code to Contribution: Documenting and Structuring Your Thesis

8.1 The Reproducibility Checklist

To ensure the research is transparent, credible, and reproducible, a final archival step is necessary. Before writing the final thesis document, complete the following checklist:

- [] **Environment:** Save the final Conda environment to a file using `conda env export > environment.yaml`.
- [] **Code Versioning:** Record the exact Git commit hashes for the CLAM and TransMIL repositories that were used for the final experiments.
- [] **Data:** Archive the exact master.csv and the complete set of split files used for the k-fold cross-validation.
- [] **Artifacts:** Save all training logs, which contain the epoch-by-epoch performance, and the final trained model weights (.pth files) for each fold.
- [] **Scripts:** Ensure all analysis, from collating results to generating final plots and statistical tests, is fully scripted and can be run from a single command.

8.2 A Blueprint for Your Thesis Document

The detailed protocol and results from this project provide a direct blueprint for the core chapters of the Master's thesis document.

- **Methods:** This section should be a narrative version of Parts I through VI of this report. Describe the dataset curation, the environment setup (referencing the environment.yaml), the model architectures, the specific code adaptations made, the training protocol (hyperparameters from Tables 2 and 3), and the complete statistical evaluation plan (referencing Table 5). The level of detail should be sufficient for another researcher to replicate the study exactly.
- **Results:** This section presents the findings without interpretation.
 - **Quantitative Results:** Present the final, populated performance matrix (Table 4). Include figures showing ROC curves for the classification task and Kaplan-Meier survival plots illustrating the risk stratification achieved by each model (with log-rank p-values annotated). Report the p-values from the DeLong's tests comparing the models' AUCs.
 - **Qualitative Results:** Dedicate a subsection to the interpretability analysis. Showcase the side-by-side comparisons of attention heatmaps for the selected illustrative patient cases. For each case, provide a brief pathological context and describe the visual differences in the heatmaps.
- **Discussion:** This is the section for synthesis and interpretation. Analyze the findings from the Results section. Why might one model have outperformed the other? Did the added complexity of TransMIL's self-attention provide a statistically significant benefit for survival prediction, or did CLAM's simpler mechanism perform comparably? Connect the patterns observed in the attention heatmaps to known prognostic features in breast cancer pathology. Contextualize the findings within the existing literature and acknowledge the study's limitations (e.g., single cancer type, single feature extractor).
- **Conclusion and Future Work:** Briefly summarize the key findings and the answer to the research question posed in the introduction. Reiterate the main contribution of the thesis. Conclude by suggesting concrete directions for future research, positioning this work as a foundational step. The most logical next step is to use the best-performing MIL architecture as the vision component in a multimodal fusion framework that integrates genomic and clinical data, demonstrating a clear and ambitious path forward.

Works cited

1. Mahmood Lab – Computational Pathology – Computational and Quantitative Pathology at

Harvard, <https://faisal.ai/> 2. Mahmood Lab @ Harvard/MGB - GitHub, <https://github.com/mahmoodlab> 3. Managing Python — conda 23.1.0 documentation, <https://docs.conda.io/projects/conda/en/23.1.x/user-guide/tasks/manage-python.html> 4. Upgrade to python 3.8 using conda - Stack Overflow, <https://stackoverflow.com/questions/58568175/upgrade-to-python-3-8-using-conda> 5. Installing PyTorch with Cuda :: TELIN Helpdesk, <https://telin.ugent.be/telin-docs/windows/pytorch/> 6. How to Install Pytorch with CUDA support on Windows 10 - GPU Mart, <https://www.gpu-mart.com/blog/Installing-pytorch-with-cuda-support-on-Windows> 7. Install pytorch with Cuda 12.1, <https://discuss.pytorch.org/t/install-pytorch-with-cuda-12-1/174294> 8. Set up Gemini Code Assist for individuals - Google for Developers, <https://developers.google.com/gemini-code-assist/docs/set-up-gemini> 9. Gemini CLI + VS Code: Native diffing and context-aware workflows, <https://developers.googleblog.com/en/gemini-cli-vs-code-native-diffing-context-aware-workflows/> 10. Gemini CLI | Gemini for Google Cloud, <https://cloud.google.com/gemini/docs/codeassist/gemini-cli> 11. MahmoodLab/UNI2-h-features · Datasets at Hugging Face, <https://huggingface.co/datasets/MahmoodLab/UNI2-h-features> 12. MahmoodLab/UNI - Hugging Face, <https://huggingface.co/MahmoodLab/UNI> 13. MahmoodLab/UNI2-h - Hugging Face, <https://huggingface.co/MahmoodLab/UNI2-h> 14. Pan-Cancer Atlas (PanCanAtlas) - MSK Data Catalog, <https://datacatalog.mskcc.org/dataset/10404> 15. TCGA - PanCanAtlas Publications - NCI Genomic Data Commons - National Cancer Institute, <https://gdc.cancer.gov/about-data/publications/pancanatlas> 16. Work with multiple repositories - Visual Studio - Microsoft Learn, <https://learn.microsoft.com/en-us/visualstudio/version-control/git-multi-repository-support?view=vs-2022> 17. How to Setup and Use Multiple Git Repositories for One Single Project - ByteScout, <https://bytescout.com/blog/setup-and-use-git-repositories-for-one-project.html> 18. mahmoodlab/CLAM: Open source tools for computational pathology - Nature BME - GitHub, <https://github.com/mahmoodlab/CLAM> 19. TransMIL: Transformer based Correlated Multiple Instance Learning for Whole Slide Image Classification - GitHub, <https://github.com/szc19990412/TransMIL> 20. TransMIL: Transformer based Correlated Multiple Instance Learning for Whole Slide Image Classification, <https://proceedings.neurips.cc/paper/2021/file/10c272d06794d3e5785d5e7c5356e9ff-Paper.pdf> 21. Evaluating Survival Models — scikit-survival 0.25.0, https://scikit-survival.readthedocs.io/en/stable/user_guide/evaluating-survival-models.html 22. 02_kaplan_meier(Python) - Databricks, https://www.databricks.com/notebooks/telco-accel/02_kaplan_meier.html 23. Log Rank Test - GeeksforGeeks, <https://www.geeksforgeeks.org/r-language/log-rank-test/> 24. logrank — SciPy v1.16.2 Manual, <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.logrank.html> 25. More examples and recipes — lifelines 0.30.0 documentation, <https://lifelines.readthedocs.io/en/latest/Examples.html> 26. sksurv.compare.compare_survival — scikit-survival 0.25.0 - Read the Docs, https://scikit-survival.readthedocs.io/en/stable/api/generated/sksurv.compare.compare_survival.html 27. Comparing ROC Curves in Machine Learning Model with DeLong's Test: A Practical Guide Using Python and MLstatkit | by Yong Zhen Huang - Medium, <https://medium.com/statistics-in-machine-learning/comparing-roc-curves-in-machine-learning-model-with-delongs-test-a-practical-guide-using-python-e70b5d20abde> 28. MLstatkit - PyPI, <https://pypi.org/project/MLstatkit/> 29. Data-efficient and weakly supervised computational pathology on whole-slide images - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC8711640/>